Plan de pruebas

Nombre del proyecto	ADOPTAFACIL
Nombre del sistema	Sistema de información para la adopción de animales
Versión	1.0
Equipo	Nicolas Alberto Valenzuela Pacheco Brayan Esteban Sanchez Avila Andres Felipe Gamez Arias Midred Callejas Chaparro
Instructor	Ruben Sanabria

1. Introducción
PARTE I. Fundamentos teóricos del plan de pruebas
2. Objetivos del plan de pruebas
    2.1. Objetivo general
    2.2. Objetivos específicos
3. Alcance de las pruebas
    3.1. Componentes funcionales
    3.2. Componentes técnicos
    3.3. Componentes no funcionales
    3.4. Exclusiones del alcance
4. Tipos de pruebas
    4.1. Pruebas unitarias
    4.2. Pruebas de integración
    4.3. Pruebas de carga
    4.4. Pruebas de estrés
5. Estrategia de pruebas
    5.1. Fase 1. Preparación del entorno de pruebas
    5.2. Fase 2. Pruebas unitarias
    5.3. Fase 3. Pruebas de integración
    5.4. Fase 4. Pruebas de rendimiento
6. Herramientas de pruebas

7. Entorno de pruebas
7.1. Componentes del entorno
8. Roles y responsabilidades
9. Criterios de entrada y salida
9.1. Criterios de entrada
9.2. Criterios de salida
10. Riesgos de pruebas
11. Cronograma de pruebas
PARTE II. Plan de pruebas técnico para AdoptaFácil
12. Estructura de pruebas del proyecto
13. Pruebas unitarias planificadas
13.1. Casos de prueba previstos
13.1.1. PoliciesTest
13.1.2. SecureFileUploadTest
14. Pruebas de integración
14.1. Ejemplos de pruebas previstas
14.1.1. AuthenticationTest
14.1.2. RegistrationTest
14.1.3. DashboardTest
14.1.4. EstadisticasTest
15. Pruebas de carga
16. Pruebas de estrés
17. Automatización de pruebas
18. Resultados esperados

---

1. Introducción

El presente documento describe el **Plan de Pruebas del sistema AdoptaFácil**, una plataforma web desarrollada con el framework **Laravel** cuyo propósito es facilitar procesos relacionados con la adopción de mascotas, gestión de usuarios, donaciones y seguimiento de solicitudes.

El objetivo principal de este documento es **definir la estrategia, metodología, herramientas y actividades que se emplearán para verificar y validar la calidad del software**, asegurando que el sistema cumpla con los requisitos funcionales y no funcionales definidos durante el desarrollo.

Este plan establece las directrices para la ejecución de diferentes tipos de pruebas, incluyendo **pruebas unitarias, pruebas de integración, pruebas de**

carga y pruebas de estrés, utilizando herramientas compatibles con el ecosistema Laravel como **PestPHP, Laravel Testing y k6**.

PARTE I. Fundamentos teóricos del plan de pruebas

2. Objetivos del plan de pruebas

2.1. Objetivo general

Definir la estrategia de pruebas que permitirá verificar la calidad, confiabilidad y correcto funcionamiento del sistema AdoptaFácil antes de su despliegue en producción.

2.2. Objetivos específicos

- Establecer los **tipos de pruebas** que se aplicarán al sistema.
- Definir las **herramientas tecnológicas** utilizadas en el proceso de testing.
- Determinar los **criterios de entrada y salida** de las pruebas.
- Establecer los **roles y responsabilidades** del equipo involucrado en el proceso de pruebas.
- Identificar los **riesgos potenciales asociados a las pruebas**.
- Definir un **cronograma estimado para la ejecución de pruebas**.

3. Alcance de las pruebas

El alcance de este plan incluye la verificación de los siguientes componentes del sistema AdoptaFácil.

3.1. Componentes funcionales

- Gestión de usuarios.
- Registro y autenticación.
- Gestión de mascotas.
- Gestión de solicitudes de adopción.

- Panel de administración.
- Sistema de donaciones.
- Visualización de estadísticas.
- Sistema de favoritos.
- Gestión de refugios o shelters.
- Interacciones comunitarias.

3.2. Componentes técnicos

- Controladores del backend Laravel.
- Modelos y políticas de seguridad.
- APIs internas.
- Procesos de autenticación.
- Manejo de archivos y seguridad.
- Integridad de la base de datos.

3.3. Componentes no funcionales

- Rendimiento del sistema.
- Capacidad de respuesta bajo carga.
- Estabilidad bajo estrés.
- Seguridad en operaciones críticas.

3.4. Exclusiones del alcance

No forman parte del alcance de este plan:

- Pruebas de usabilidad formal.
- Pruebas manuales de experiencia de usuario.
- Auditorías externas de seguridad.

4. Tipos de pruebas

Durante el proceso de validación del sistema se aplicarán diferentes tipos de pruebas con el fin de evaluar distintos aspectos del software.

4.1. Pruebas unitarias

Las pruebas unitarias tienen como objetivo validar el funcionamiento correcto de **componentes individuales del sistema**, como funciones, clases o métodos específicos. Estas pruebas permitirán detectar errores tempranos en la lógica del código.

Ejemplos de componentes a evaluar:

- Políticas de autorización.
- Validaciones de seguridad.
- Funciones internas del sistema.

Herramienta utilizada: PestPHP.

4.2. Pruebas de integración

Las pruebas de integración permiten verificar la correcta interacción entre los diferentes componentes del sistema.

Estas pruebas evaluarán:

- Interacción entre controladores y modelos.
- Procesos de autenticación.
- Flujos de registro de usuarios.
- Operaciones CRUD.

Herramienta utilizada: Laravel Testing Framework.

4.3. Pruebas de carga

Las pruebas de carga permiten medir el comportamiento del sistema cuando múltiples personas usuarias acceden simultáneamente a la plataforma.

Estas pruebas evaluarán:

- Tiempo de respuesta del servidor.
- Capacidad de manejo de múltiples solicitudes.
- Consumo de recursos del sistema.

Herramienta utilizada: k6.

4.4. Pruebas de estrés

Las pruebas de estrés permiten evaluar el comportamiento del sistema cuando se somete a cargas superiores a las esperadas. El objetivo es identificar:

- Límites del sistema.
- Posibles cuellos de botella.
- Fallos bajo condiciones extremas.

Herramienta utilizada: k6.

5. Estrategia de pruebas

La estrategia de pruebas adoptada para el sistema AdoptaFácil seguirá un enfoque **incremental y automatizado**, priorizando la validación temprana del código.

La estrategia se divide en cuatro fases.

5.1. Fase 1. Preparación del entorno de pruebas

- Configuración del entorno Laravel.
- Configuración de base de datos de testing.
- Instalación de herramientas de pruebas.
- Configuración de scripts de ejecución.

5.2. Fase 2. Pruebas unitarias

Se ejecutarán pruebas sobre componentes individuales del sistema.

Objetivo: validar la lógica interna del sistema.

5.3. Fase 3. Pruebas de integración

Se evaluará la interacción entre los distintos módulos del sistema.

Objetivo: verificar que los flujos del sistema funcionen correctamente.

5.4. Fase 4. Pruebas de rendimiento

Se realizarán pruebas de carga y estrés para evaluar la estabilidad del sistema.

Objetivo: determinar la capacidad del sistema bajo diferentes condiciones de tráfico.

6. Herramientas de pruebas

Las siguientes herramientas serán utilizadas durante el proceso de pruebas.

Herramienta	Tipo de pruebas	Descripción
PestPHP	Pruebas unitarias	Framework moderno de testing para PHP basado en PHPUnit
Laravel Testing	Pruebas de integración	Sistema de testing integrado en Laravel
k6	Pruebas de carga	Herramienta de pruebas de rendimiento
k6	Pruebas de estrés	Evaluación de comportamiento bajo cargas extremas
GitHub Actions	Automatización	Ejecución automática de pruebas en CI/CD

7. Entorno de pruebas

El entorno de pruebas se configurará para simular las condiciones reales de operación del sistema.

7.1. Componentes del entorno

Servidor de aplicación:

- Laravel Framework.

- PHP 8+.

Base de datos:

- MySQL o MariaDB.

Infraestructura:

- Contenedor Docker para despliegue.
- Entorno de integración continua mediante GitHub Actions.

Sistema operativo recomendado:

- Linux o entorno compatible con Docker.

8. Roles y responsabilidades

Rol	Responsabilidad
Desarrollador	Implementar y ejecutar pruebas unitarias
Ingeniero de pruebas	Diseñar y ejecutar pruebas de integración
Ingeniero DevOps	Configurar pipelines de testing
Líder de proyecto	Validar resultados de pruebas

9. Criterios de entrada y salida

9.1. Criterios de entrada

Las pruebas podrán comenzar cuando:

- El código fuente esté disponible en el repositorio.
- El entorno de pruebas esté configurado.
- Las dependencias del sistema estén instaladas.
- Los scripts de prueba estén definidos.

9.2. Criterios de salida

Las pruebas se considerarán satisfactorias cuando:

- Todas las pruebas unitarias pasen correctamente.

- Las pruebas de integración no presenten errores críticos.
- El sistema soporte la carga definida sin degradación significativa.
- Los errores encontrados hayan sido documentados.

10. Riesgos de pruebas

Durante el proceso de pruebas pueden presentarse diferentes riesgos.

Riesgo	Descripción
Entorno incorrecto	Configuración distinta a producción
Datos de prueba insuficientes	Resultados no representativos
Fallos de infraestructura	Problemas en servidores de pruebas
Cambios de código	Alteraciones durante la ejecución de pruebas

11. Cronograma de pruebas

Fase	Actividad	Duración estimada
1	Preparación del entorno	2 días
2	Pruebas unitarias	3 días
3	Pruebas de integración	3 días
4	Pruebas de carga	2 días
5	Pruebas de estrés	2 días
6	Documentación de resultados	2 días

Duración total estimada: 14 días.

PARTE II. Plan de pruebas técnico para AdoptaFácil

12. Estructura de pruebas del proyecto

El proyecto AdoptaFácil implementará una estructura de pruebas organizada dentro del directorio `tests/`.

Estructura planificada:

```
tests
├── Feature
│   ├── Auth
│   ├── DashboardTest
│   ├── EstadisticasTest
│   └── Settings
│
├── Unit
│   ├── PoliciesTest
│   └── SecureFileUploadTest
│
├── Performance
│   ├── k6-load.js
│   ├── k6-stress.js
│   └── k6-smoke.js
```

13. Pruebas unitarias planificadas

Las pruebas unitarias se desarrollarán utilizando **PestPHP**.

13.1. Casos de prueba previstos

13.1.1. PoliciesTest

Objetivo: validar que las políticas de autorización del sistema funcionen correctamente.

Escenarios previstos:

- Acceso permitido a usuarios autorizados.
- Bloqueo de acceso a usuarios no autorizados.

13.1.2. SecureFileUploadTest

Objetivo: validar el manejo seguro de archivos subidos por las personas usuarias.

Escenarios previstos:

- Validación de extensiones permitidas.
- Rechazo de archivos peligrosos.
- Protección contra cargas maliciosas.

14. Pruebas de integración

Las pruebas de integración verificarán el funcionamiento de flujos completos del sistema.

14.1. Ejemplos de pruebas previstas

14.1.1. AuthenticationTest

Validará:

- Inicio de sesión.
- Validación de credenciales.
- Redirección del usuario autenticado.

14.1.2. RegistrationTest

Validará:

- Registro de nuevos usuarios.
- Validación de datos.
- Creación correcta del usuario.

14.1.3. DashboardTest

Validará:

- Acceso al panel principal.
- Carga correcta de información.

14.1.4. EstadisticasTest

Validará:

- Generación de estadísticas.
- Integridad de los datos mostrados.

15. Pruebas de carga

Las pruebas de carga se ejecutarán utilizando **k6**.

Script planificado: `k6-load.js`.

Escenario planificado:

- Simulación de múltiples usuarios accediendo al sistema.
- Evaluación de tiempos de respuesta.
- Monitoreo del rendimiento del servidor.

Métricas a evaluar:

- Tiempo promedio de respuesta.
- Tasa de errores.
- Número de solicitudes por segundo.

16. Pruebas de estrés

Las pruebas de estrés se ejecutarán mediante el script `k6-stress.js`.

Objetivo:

- Determinar el límite máximo de usuarios concurrentes.
- Evaluar la estabilidad del sistema bajo condiciones extremas.

Indicadores evaluados:

- Saturación del servidor.
- Tiempo de respuesta elevado.
- Fallos del sistema.

17. Automatización de pruebas

El sistema contará con integración continua mediante **GitHub Actions**.

Las pruebas se ejecutarán automáticamente cuando:

- Se realice un push al repositorio.
- Se genere un pull request.

El pipeline incluirá:

1. Instalación de dependencias.
2. Configuración del entorno.
3. Ejecución de pruebas unitarias.
4. Ejecución de pruebas de integración.
5. Generación de reportes.

18. Resultados esperados

Al finalizar el proceso de pruebas se espera que:

- El sistema funcione correctamente en todos los escenarios evaluados.
- Los errores críticos hayan sido corregidos.
- El sistema soporte cargas normales de usuarios.
- El software esté listo para su despliegue en producción.